LamTools：具备能力感知委派机制的本地优先 Agent 运行时

张玉霖 与 张一明 - 共同第一作者
独立研究员

中文主版本 / Chinese-primary version - Technical Paper / Preprint - LamTools v0.2.6 - 审计 2026-08-20；全量复核 2026-08-21
Software DOI：10.5281/zenodo.22039646
Paper DOI：10.5281/zenodo.22040870

摘要

大语言模型应用正在同时组合异构模型、工具、文件和持久化状态。LamTools 是一个本地优先 Agent 运行时，将其中一条关键路径显式化：父 Agent 可以调用具有稳定名称的子会话，解析可选模型覆盖，根据子模型声明的能力转换附件，并将子会话事件与结果投影回父时间线。运行时限制递归调用和工具可见范围，同时在 SQLite 中保存审批、取消和检查点状态。在经过审计的 v0.2.6 基线下，完整 Core 测试套件包含 1,484 项；最近一次完整重跑记录为通过 1,481 项、跳过 2 项，并有 1 项对时间阈值敏感的失败，单独重跑该用例时通过。刷新后的定向机制套件记录为通过 63 项、跳过 1 项。另有一个 14 问、8 文档的检索案例，对比 BM25-only 与本地嵌入加混合检索。本地混合路径将任一黄金文档 Recall@10 从 0.857 提升到 1.000，将首个任一黄金文档 MRR 从 0.821 提升到 0.917，同时使 warm query-time P95 延迟从 1.75 ms 增加到 6.67 ms。本文将这些结果作为绑定版本的工程证据和面向复现的系统描述，不将其包装为新的路由算法或模型优越性结论。

关键词： AI agents；模型委派；多模态能力；持久执行；本地优先软件；检索评估；可复现性

1. 引言

Agent 系统已经不再只是一次模型调用。实际运行时需要同时协调模型回合、工具、文件、审批、打断和持久化状态，并让操作者能够检查和控制任务。当配置中的模型能力不一致时，问题会进一步扩大。纯文本模型可能适合规划和语言推理，而另一个模型才适合解释图像或其他附件类型。如果把这种差异隐藏在提供商适配器内部，执行路径就很难测试，也很难复现。

LamTools 将这个问题作为桌面端和命令行工作流中的本地运行时来处理。当前产品主体是 core/ 包，它为 Vue 工作台、CLI 和 HTTP/WebSocket 客户端提供共同循环。运行时状态、事件投影、检查点和调度记录保存在本地 SQLite 中。当用户配置远程提供商时，提供商调用仍然可能离开本机；"本地优先"描述的是编排状态的所有权，并不承诺每一次模型调用都在本地完成 [9]。

本文研究已经实现的委派路径，而不是把产品中的每个功能都包装成独立研究贡献。该路径包含四个可观察阶段：父 Agent 请求子任务；运行器解析有效模型和执行模式；附件内容按照子模型声明的能力拆分；子时间线通过父事件表面返回。相同的运行时还提供受边界约束的工具、审批继续和基于检查点的恢复，这些机制是委派在受操作者控制的应用中真正可用的必要条件。

本文的贡献保持窄而有证据边界。第一，记录一种将命名子会话、显式模型选择、能力感知附件处理、权限约束工具和本地持久状态组合起来的运行时设计。这里最值得强调的设计信号是：模型能力被当作运行时契约，而不是 prompt 约定；它直接控制附件转换，并在最容易发生静默多模态失败的边界上接受测试。第二，使用确定性测试而不是未报告的提供商 benchmark，对实现机制进行行为评估。委派被当作有范围的持久执行，并在已测试的运行时组件之间检查子会话身份、权限、审批状态、事件投影和检查点范围。第三，报告一个保存逐问题输出的小型检索案例，使质量与延迟之间的具体权衡可以被检查。几个选择共同把一组看似普通的 Agent 功能提炼成可审计的工程系统描述。本文不声称优先于通用多 Agent 框架、学习型模型路由器、记忆算法或持久图系统。

2. 结果

2.1. 运行时架构与委派路径

经过审计的运行时由五个协作层组成。客户端层提交回合并渲染事件。Core Loop Kernel 协调模型回合、工具准备、审批和等待状态、重试、终止收敛与事件持久化。工具箱与插件层在明确的模式和权限过滤下暴露原生、MCP 及插件工具。子 Agent 层创建或恢复命名子会话，解析模型引用，关闭递归 sub_agent 调用并转发子事件。持久化层保存线程、事件、检查点、回滚/分叉图数据和 Arrange 调度记录。

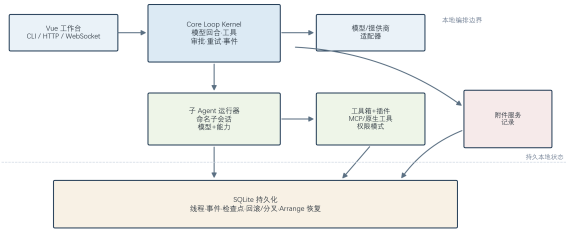

图 1。审计基线 v0.2.6 的 LamTools 运行时架构。实线边界表示本地编排状态；当操作者如此配置时，提供商调用仍可能是远程的。

委派工具接受任务、稳定的子会话名称、可选模型引用、可选工具模式和附件标识符。模型引用可以是已配置的模型 ID，也可以是显示名称。运行器在能力查询和后续元数据构造前先将其规范化为 canonical ID。这一顺序很重要：能力判断必须针对有效模型，而不能针对含义不明确的显示标签。

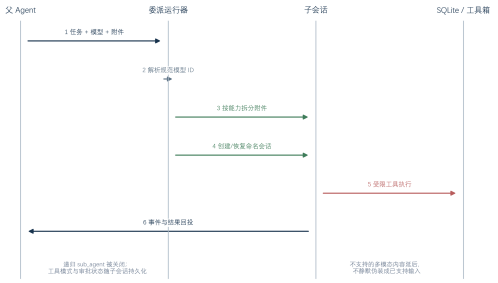

图 2。已实现的委派路径时序。能力转换、子会话持久化和受边界约束的工具执行都是显式运行时步骤，而不是隐藏在提供商内部的行为。

表 1 将可观察的运行机制与不作出的结论分开。这一边界本身就是结果的一部分：如果把计划中或历史版本的功能无提示地混入当前基线，系统论文就会变得不可复现。

运行时表面	v0.2.6 的已实现行为	本文使用的证据	结论边界
核心循环	模型回合、工具准备、审批/等待、重试、终止状态和事件持久化	完整 Core 套件	运行时行为，不是模型质量优越性
子会话	命名会话创建/复用、可选模型覆盖、子事件转发	委派定向测试	显式委派原语，不是学习型路由器
附件处理	文本保持可用；支持的多模态内容变成内容块；不支持内容延后处理	附件能力测试	能力感知转换，不是通用多模态支持
工具边界	关闭子递归；执行/只读模式和允许列表生效	子 Agent 与工具箱测试	有界权限，不是完整安全证明
持久状态	SQLite 线程/事件、检查点、回滚/分叉、取消和 Arrange 恢复	检查点、持久化和调度测试	已实现恢复路径，不是新的持久化理论
RAG 插件	BM25/FTS5 回退、可选本地向量和混合检索	保存的 14 问检索报告	小型案例，不是通用 benchmark

2.2. 能力感知的附件处理

对于每个附件，运行时取得已保存的附件记录，并读取有效子模型的配置能力。支持的内容会作为模型内容块发出。文本附件仍作为文本可用。不支持的多模态内容按照附件服务契约延后，而不是静默地伪装成子模型已经能够理解的输入。这样，能力边界被放入可以直接测试的运行时组件中，而不是散落在 UI 中的提供商判断。

子内核使用与父内核相同的核心循环，但关闭递归 sub_agent 调用。调用者可以选择只读模式、完整执行模式或显式允许的工具集合。如果工具需要审批，子会话可以进入等待状态；审批继续会先把工具结果写入持久化的子历史，再恢复执行。子事件携带父标识符，使 UI 能够展示委派工作，同时不把子时间线与父回合混为一谈。

这一设计应当被作出适度解释。它是使异构模型使用过程显式、可观察的集成机制。它不会学习哪一个模型最优，不会估计通用能力分数，也没有证明委派会在任意提供商上提升回答质量。

2.3. 持久状态、控制与恢复

LamTools 在模型回合周围创建检查点，并将检查点关系表示成图。恢复操作可以选择范围，例如只恢复子对话而不重写父对话。分叉操作创建新会话，同时保留指向源检查点的图边。相同的状态边界也支持取消持久化和审批继续，因此被打断或正在等待的子会话不会退化成内存中的回调。

Arrange 增加定时和事件触发任务。运行时记录发生记录，处理暂停和取消转换，回收过期租约，并在启动恢复阶段检查停滞的活动回合。这些能力之所以与委派相关，是因为桌面运行时中的委派不只是一次函数调用：操作者必须能够停止、检查、恢复或分支任务。测试使用确定性存储和时钟验证这些转换，但不声称具备生产规模的容错能力。

2.4. 绑定版本的评估总览

评估将确定性的运行时行为与检索质量分开。第一道门检查实现是否保持声明的状态和权限转换；第二道门是在固定语料和黄金问题上的小型质量案例。图 3 展示记录的测试数量以及推导得到的检索分数差异，但不把两类证据合并成一个总分。

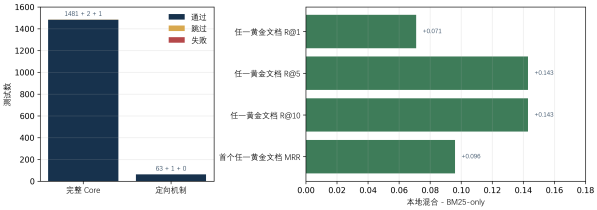

图 3。评估总览。左图报告记录的测试数量；右图报告保存的检索指标中本地混合减去 BM25-only 的差值。

本节所有数值对应 LamTools v0.2.6、tag v0.2.6、commit dd54b7fdd57f4eb0926f1dd3a94fbf2c2bb0fd8a。最近一次正确配置的完整 Core 运行收集 1,484 项测试，记录为通过 1,481 项、跳过 2 项、1 项时间阈值敏感的失败和 3 个警告；完整输出保存在 docs/paper/evidence/core-pytest-v0.2.6-20260820.log，该失败用例单独重跑日志保存在 docs/paper/evidence/timing-sensitive-isolated-v0.2.6-20260821.log。刷新后的定向机制运行覆盖委派、命名会话、模型覆盖、能力处理、检查点图操作、回滚/分叉以及 Arrange 调度/恢复；记录为通过 63 项、跳过 1 项，日志保存在 docs/paper/evidence/targeted-mechanism-v0.2.6-20260821.log。定向运行是机制探针，不能替代完整套件。

证据流	分析单位	记录结果	解释
完整 Core 套件	仓库测试用例	通过 1,481；跳过 2；时间阈值失败 1	广泛回归证据，但仍有一个环境敏感测试未解决
运行时定向套件	委派、持久化和调度案例	通过 63；跳过 1	直接支撑本文机制的行为证据
检索案例	8 个文档、14 个问题、两种检索模式	每种模式 14 条逐问题记录	可检查的小型质量/延迟对比
可复现目标	发布与源代码状态	v0.2.6 / dd54b7f...	防止把后续代码与报告数值混合

2.5. 检索案例与工程权衡

仓库包含 8 个中文合同文档和 14 问黄金集合，问题类型包括精确、数字、改写和多文档问题。保存的对比是 BM25-only 与本地嵌入加混合检索。本地混合检索使任一黄金文档 Recall@1 增加 0.071，使 Recall@5 和 Recall@10 各增加 0.143，使首个任一黄金文档 MRR 增加 0.096。Warm query-time P95 延迟增加 4.92 ms，在记录环境中约为 BM25-only 的 3.8 倍。

配置	任一黄金文档 Recall@1	任一黄金文档 Recall@5	任一黄金文档 Recall@10	首个任一黄金文档 MRR	Warm query P95
BM25-only	0.786	0.857	0.857	0.821	1.75 ms
本地嵌入 + 混合检索	0.857	1.000	1.000	0.917	6.67 ms
差值	+0.071	+0.143	+0.143	+0.096	+4.92 ms

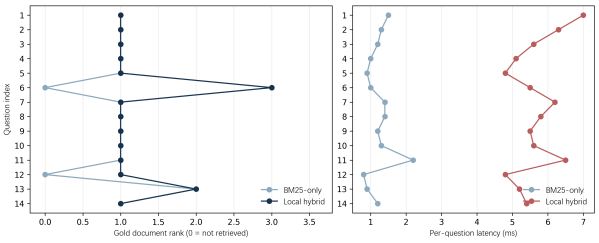

图 4。保存检索报告中的逐问题证据。排名 0 表示没有检索到黄金文档；延迟图保留实际观测的墙钟值，而不是拟合分布。

逐问题视图可以防止聚合结果造成误导。BM25-only 未检索到多文档仲裁问题和"押金什么时候退还"的改写问题，而本地混合检索对两者都返回了非零排名的黄金文档。混合路径也改变了部分顶部文档顺序，并增加了每一个记录问题的延迟。因此，这个案例只支持一个窄的工程结论：本地向量腿在该语料上提高了覆盖率，同时产生了可测量的本地延迟成本。它不支持法律回答正确性、大规模检索质量或跨提供商性能结论。

3. 讨论

最值得强调的技术信号并不是 LamTools 能够调用另一个模型，而是它把这次调用变成可解释的能力契约和持久范围。有效模型、附件能力、子会话身份、工具模式、审批状态和父投影都可以分别检查、测试和恢复。这一边界提供了一个运行时防护，可以在问题被误读成模型失败以前暴露一类静默的多模态不匹配；但本文没有测量提供商失败率下降。代价是比单次模型调用需要更多状态协调，并且拥有更大的故障表面。子会话可能等待审批、需要恢复，或者携带当前模型无法消费的内容；显式表达这些情况提升可观察性，但不会消除它们。

检索案例从另一层面展示了同一权衡。加入本地向量腿提升了小型语料的首个命中覆盖率，但 warm query-time P95 延迟从 1.75 ms 增加到 6.67 ms。正确结论不是"混合检索在抽象意义上更好"，而是运行时可以暴露一个可测量的质量/延迟选择，并保留逐问题证据以便重新评估。

对软件型 Agent 系统而言，可信论文应当说明哪些行为已经实现、哪些仍然计划中、基线固定了什么，以及证据在哪里停止。LamTools 的最强信号正是能力处理、持久控制和可量化权衡之间的可审计连接，而不是本地运行时或委派策略普遍优越的证据。

4. 相关工作

AutoGen [1] 和 AgentScope [5] 等通用多 Agent 框架与平台为构建多 Agent 应用提供抽象和系统支持。LamTools 在 Agent 组合层面存在重叠，但提出的运行时主张更窄：子 Agent 是具有稳定身份、受限递归和显式模型/能力处理的持久工具调用。本文不将任务质量与这些平台进行比较，也不声称提出新的多 Agent 协议。

FrugalGPT 研究跨语言模型的级联、成本和质量 [2]，RouteLLM 则利用偏好数据学习模型路由器 [3]。这些工作说明了异构模型使用的研究背景，但 LamTools 不训练路由器、不学习级联策略，也不优化成本目标。本文证据针对的是一条显式执行路径：操作者或配置调用可以

选择子模型，并把后果保存在本地运行时状态中。

MemGPT 将上下文管理和打断视为类似操作系统的记忆问题 [4]。LangGraph 文档介绍检查点保存器、线程和持久图状态 [8]。LamTools 同样关注可恢复执行，但本文证据针对的是自己的 SQLite 检查点图、审批继续、子会话状态和回滚/分叉操作。MCP 为 LLM 应用中的 prompts、resources 和 tools 提供标准规范 [7]；LamTools 将 MCP 集成进更大的本地、审批感知工具箱，同时保留原生工具和插件范围工具。AgentDojo 研究工具调用 Agent 在 prompt injection 下的安全性 [6]；LamTools 实现本地审批和可见性边界，但不提供安全证明或对抗鲁棒性评估。

本文对"本地优先"的使用基于 Kleppmann 等人关于本地所有权、可用性和用户控制的原则 [9]。LamTools 将这一框架用于桌面运行时的编排状态，并不声称所有配置的提供商调用都在本地完成。

相关路线	已有工作提供的内容	LamTools 的边界
多 Agent 平台	Agent 与平台抽象 [1, 5]	运行时级子会话操作
模型路由	成本/质量选择与学习型路由器 [2, 3]	显式选择路径；没有学习型优化器
有状态 Agent	上下文管理与打断概念 [4]	SQLite 运行时状态与检查点图
持久图	检查点、线程和保存器 [8]	本地审批、子范围、回滚/分叉
工具安全与互操作	工具协议与对抗性评估 [6, 7]	本地权限下的 MCP 加原生/插件工具；不提供安全证明
本地优先软件	本地所有权与用户控制原则 [9]	本地编排状态，不保证提供商本地化

5. 方法

5.1. 审计与版本控制

证据边界通过阅读 v0.2.6 的活动 core/ 包、测试、发布工作流、配置和文档建立。归档产品和仅存在于路线图中的功能被排除。每一个面向论文的结果都绑定到 tag v0.2.6 和 commit dd54b7fdd57f4eb0926f1dd3a94fbf2c2bb0fd8a；仓库审计于 2026 年 8 月 20 日完成，带日志的完整重跑于 2026 年 8 月 21 日完成。完整 Core 运行使用临时的 LAMTOOLS_COMMAND_SHELL=pwsh 选择，因为默认 Git Bash 环境无法解析 Windows Python 命令；这是环境选择细节，不是本机没有 Python。

5.2. 确定性运行时评估

运行时评估使用伪造的确定性模型客户端和临时 SQLite 工作区。测试断言状态转换和事件投影，而不是要求提供商生成主观回答。定向集合覆盖：规范模型 ID 解析；命名子会话复用；模型覆盖；多模态转发和文本模型延后处理；工具模式和允许列表；递归委派阻断；审批继续；取消持久化；父子事件投影；检查点图创建；范围恢复；回滚；分叉；定时执行；暂停/取消转换；租约回收；以及启动恢复。

完整套件和定向套件的数量按照记录结果报告，跳过项保持可见。没有从面向论文的证据中删除失败测试，也没有用提供商分数替换确定性行为断言。

5.3. 检索协议

检索案例使用仓库保存的 14 问黄金集合、8 文档中文合同语料和两个报告文件：e2e/rag-eval/reports/retrieval-none-20260815-095234.json 与 e2e/rag-eval/reports/retrieval-local-20260815-101253.json。两种模式使用相同的问题和黄金文档标注。报告保留逐问题排名、顶部文档标识符、适用时的向量命中数量以及墙钟延迟。面向论文的派生结果保存在 docs/paper/evidence/retrieval-derived-v0.2.6.json，由 docs/paper/derive_retrieval_metrics.py 生成，不改写原始报告。

对于大小为 (N) 的问题集合，任一黄金文档 Recall@k 是前 k 个去重文档结果中出现任一黄金文档的问题比例。多文档问题只要出现任一黄金文档就计为命中，因此本文不测量完整集合召回率。MRR 为 (N^{-1}sum_i 1/r_i)，其中 (r_i) 是第一个任一黄金文档的排名；没有排名时贡献 0。Warm query-time P95 使用保存的逐问题时间，并采用 inclusive 线性插值计算。报告的差值是派生聚合值之间的直接算术差；由于案例规模小且保存报告没有重复独立运行，不声称置信区间或显著性检验。

5.4. 提供商 smoke test 协议

论文主张保持确定性，但另行准备了针对 OpenCode Go endpoint 的提供商 smoke test [11]。该 endpoint 兼容 OpenAI 格式，deepseek-v4-flash 和 mimo-v2.5 使用 /chat/completions。两个模型接收相同的固定 prompt，temperature 为 0，并设置较小的输出上限。smoke test 只记录模型、HTTP 状态、响应形状、延迟和提供商返回的 usage；它不会保留生成文本或凭据。这些结果最多只能作为实现连通性检查，不能作为 benchmark 或模型质量评估。

5.5. 可复现性与发布状态

可复现材料包括论文源文件、参考文献、图表生成器、生成的图、评估工具、黄金问题、原始 JSON 报告和方法说明。软件目标是现有公开 GitHub Release v0.2.6；不修改历史 tag。提供商凭据、私有配置和用户数据均排除在外。Paper Record 应归档一个中文主版本/英文完整版本合并的 PDF，Software Record 则单独归档现有 Release。

6. 局限性与不作出的结论

评估规模较小，并且主要是确定性的。没有进行提供商支持的模型质量比较、外部用户研究或大规模 benchmark。检索语料只有 8 个文档和 14 个问题，观察值不应推广到法律检索、其他语言或更大语料。报告文件没有完整的机器环境指纹，因此数值属于绑定版本的案例证据。提供商行为、API 延迟、模型版本、网络条件和本地硬件都会改变结果。

部分 RAG 会话历史路径已经存在，但旧的集成检查仍保留过时预期，因此没有把它当作完全验证的能力。计划中的 VLM 格式路由、完整表格抽取和大规模合同工作流均被排除。系统在概念上与既有 Agent 运行时、路由工作、持久化系统和工具协议重叠；本文不声称优先权或"第一"。最后，本文没有经过同行评审，双作者共同第一作者表述反映作者共识，而不是任何出版 venue 的编辑决定。

7. 数据与代码可用性

LamTools 位于 https://github.com/Lam-Arc/LamTools。精确可复现目标是公开的软件 Release v0.2.6 [10]、commit dd54b7fdd57f4eb0926f1dd3a94fbf2c2bb0fd8a。RAG 工具、语料、黄金问题和原始报告都在该 Release 的 e2e/rag-eval/ 下。冻结的软件 Release 已通过 Software DOI https://doi.org/10.5281/zenodo.22039646 归档；本双语论文记录的 Paper DOI 为 https://doi.org/10.5281/zenodo.22040870。

8. AI 辅助披露

本工作准备与审阅过程中使用了生成式 AI 工具作为辅助；研究方向、技术决策、结果解释和最终审阅均由人类作者主导。

9. 结论

LamTools 是一个已经实现的本地优先 Agent 运行时，其最有证据支撑的论文主线是：具备能力感知的子 Agent 委派与持久状态、明确工具边界和操作者控制的集成。在固定 v0.2.6 基线下，确定性测试使委派和恢复行为可检查，保存的检索案例则暴露了具体的质量/延迟权衡。因此，本文的价值是具有可见证据边界的可复现系统描述，而不是夸大的通用路由方法结论。未来版本可以扩大提供商评估和外部使用证据；当前记录只要持续明确版本、局限性和未同行评审状态，就保持有效。

参考文献

1. Wu, Q., Bansal, G., Zhang, J., et al. (2023). AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. arXiv:2308.08155. https://doi.org/10.48550/arXiv.2308.08155
2. Chen, L., Zaharia, M., and Zou, J. (2023). FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance. arXiv:2305.05176. https://doi.org/10.48550/arXiv.2305.05176
3. Ong, I., Almahairi, A., Wu, V., et al. (2024). RouteLLM: Learning to Route LLMs with Preference Data. arXiv:2406.18665. https://doi.org/10.48550/arXiv.2406.18665
4. Packer, C., Wooders, S., Lin, K., et al. (2023). MemGPT: Towards LLMs as Operating Systems. arXiv:2310.08560. https://doi.org/10.48550/arXiv.2310.08560
5. Gao, D., Li, Z., Pan, X., et al. (2024). AgentScope: A Flexible yet Robust Multi-Agent Platform. arXiv:2402.14034. https://doi.org/10.48550/arXiv.2402.14034
6. Debenedetti, E., Zhang, J., Balunovic, M., et al. (2024). AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents. arXiv:2406.13352. https://doi.org/10.48550/arXiv.2406.13352
7. Model Context Protocol. (2025). Model Context Protocol Specification, protocol revision 2025-06-18. https://modelcontextprotocol.io/specification/2025-06-18/server/index
8. LangChain. (n.d.). LangGraph Persistence and Checkpointing. Official documentation. https://docs.langchain.com/oss/python/langgraph/persistence
9. Kleppmann, M., Wiggins, A., van Hardenberg, P., and McGranaghan, M. (2019). Local-first software: You own your data, in spite of the cloud. Proceedings of Onward! '19, 154-178. https://doi.org/10.1145/3359591.3359737
10. Zhang, Yulin, and Zhang, Yiming. (2026). LamTools (Version v0.2.6) [Computer software]. Zenodo. https://doi.org/10.5281/zenodo.22039646
11. OpenCode. (n.d.). Go documentation: endpoints, models and compatibility. Official documentation. https://opencode.ai/docs/go/

LamTools: A Local-First Agent Runtime with Capability-Aware Delegation

Yulin Zhang and Yiming Zhang - Equal contribution
Independent Researcher

English version - Technical Paper / Preprint - LamTools v0.2.6 - audited 20 August 2026; full rerun 21 August 2026
Software DOI: 10.5281/zenodo.22039646
Paper DOI: 10.5281/zenodo.22040870

Abstract

Large-language-model applications increasingly combine heterogeneous models, tools, files and durable state. LamTools is a local-first agent runtime that makes one such combination explicit: a parent agent can invoke a named child session, resolve an optional model override, convert attachments according to the child model's declared capability, and project the child's events and result back into the parent timeline. The runtime bounds recursion and tool visibility while preserving approval, cancellation and checkpoint state in SQLite. At the audited v0.2.6 baseline, the complete Core suite contains 1,484 tests; the latest full rerun recorded 1,481 passed, 2 skipped and one timing-sensitive failure, while the isolated rerun of that case passed. A refreshed targeted mechanism suite recorded 63 passed and 1 skipped. A separate 14-question, 8-document retrieval case study compares BM25-only with local embedding plus hybrid retrieval. The hybrid path increases any-gold document Recall@10 from 0.857 to 1.000 and first-any-gold MRR from 0.821 to 0.917, while warm query-time p95 latency increases from 1.75 to 6.67 ms. We present these findings as version-bound engineering evidence and a reproducibility-oriented system description, not as a new routing algorithm or a claim of model superiority.

Keywords: AI agents; model delegation; multimodal capability; durable execution; local-first software; retrieval evaluation; reproducibility

1. Introduction

Agent systems are no longer defined only by a model call. A practical runtime must coordinate model turns, tools, files, approvals, interruptions and persistent state while leaving the operator able to inspect and control the task. The difficulty is amplified when configured models do not have identical capabilities. A text-oriented model may be adequate for planning and language reasoning, while a different model is needed to interpret an image or another attachment type. Treating that difference as an implicit provider detail makes the execution path difficult to test and reproduce.

LamTools addresses this problem as a local runtime for desktop and command-line workflows. The active product is the core/ package, which exposes a shared loop to the Vue workbench, CLI and HTTP/WebSocket clients. Its runtime state, event projections, checkpoints and scheduling records are stored locally in SQLite. Provider calls can still leave the machine when a remote provider is configured; "local-first" describes ownership of orchestration state, not a promise that every model call is local [9].

This paper studies the implemented delegation path rather than treating every product feature as a separate research contribution. The path has four observable stages: the parent requests a child task; the runner resolves the effective model and execution mode; attachment content is split according to the child's declared capability; and the child timeline is returned through the parent event surface. The same runtime also supplies bounded tools, approval continuation and checkpoint-based recovery, which are necessary for the delegation path to be usable in an operator-controlled application.

The contribution is deliberately narrow and evidence-bound. First, we document a runtime composition that joins named child sessions, explicit model selection, capability-aware attachment handling, permission-bounded tools and durable local state. The key design signal is that model capability is treated as a runtime contract: it directly controls attachment conversion and is tested at the boundary where silent multimodal failures would otherwise occur. Second, we provide a behavioral evaluation of the implemented mechanism using deterministic tests rather than an unreported provider benchmark. Delegation is treated as scoped durable execution across the tested runtime components, with child identity, permissions, approval state, event projection and checkpoint scope examined together. Third, we report a small retrieval case study with stored per-question outputs to make a concrete quality/latency trade-off inspectable. Together, these choices turn an otherwise ordinary collection of agent features into an auditable engineering system description. The work does not claim priority over general multi-agent frameworks, learned model routers, memory algorithms or durable graph systems.

2. Results

2.1. Runtime architecture and the delegation path

The audited runtime has five cooperating layers. The client layer submits turns and renders events. The Core Loop Kernel coordinates model rounds, tool preparation, approval and wait states, retries, terminal convergence and event persistence. The toolbox and plugin layer exposes native, MCP and plugin tools under explicit mode and permission filters. The sub-agent layer creates or resumes a named child session, resolves a model reference, disables recursive sub_agent calls and forwards child events. The persistence layer stores threads, events, checkpoints, rollback/fork graph data and Arrange scheduling records.

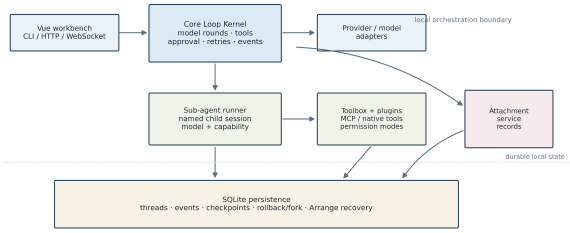

Figure 1. LamTools runtime architecture at the audited v0.2.6 baseline. The solid boundary marks local orchestration state; provider calls may be remote when configured by the operator.

The delegation tool accepts a task, a stable child-session name, an optional model reference, an optional tool mode and attachment identifiers. A model reference may be a configured model ID or a display name. The runner canonicalizes that reference before capability lookup and downstream metadata construction. This ordering is important: capability decisions must be made against the effective model, not an ambiguous display label.

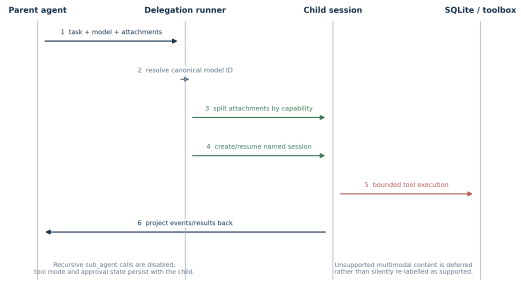

Figure 2. Sequence of the implemented delegation path. Capability conversion, child-session persistence and bounded tool execution are explicit runtime steps rather than hidden provider behavior.

Table 1 separates observable mechanisms from claims that are not made. This boundary is part of the result: a system paper becomes less reproducible when planned or historical features are silently mixed with the active baseline.

Runtime surface	Implemented behavior at v0.2.6	Evidence used here	Claim boundary
Core loop	Model rounds, tool preparation, approval/wait, retries, terminal state and event persistence	Full Core suite	Runtime behavior, not model-quality superiority
Child sessions	Named session creation/reuse, optional model override, child event forwarding	Targeted delegation tests	Explicit delegation primitive, not a learned router

Runtime surface	Implemented behavior at v0.2.6	Evidence used here	Claim boundary
Attachment handling	Text remains available; supported multimodal content becomes blocks; unsupported content is deferred	Attachment capability tests	Capability-aware conversion, not universal multimodal support
Tool boundary	Child recursion disabled; read/execute modes and allow-lists are enforced	Sub-agent and toolbox tests	Bounded permissions, not a complete security proof
Durable state	SQLite threads/events, checkpoints, rollback/fork, cancellation and Arrange recovery	Checkpoint, persistence and scheduling tests	Implemented recovery paths, not a new persistence theory
RAG plugin	BM25/FTS5 fallback, optional local vectors and hybrid retrieval	Stored 14-question retrieval reports	Small case study, not a general benchmark

2.2. Capability-aware attachment handling

For each attachment, the runtime obtains the stored attachment record and reads the configured capability of the effective child model. Supported content is emitted as a model content block. Text attachments remain available as text. Unsupported multimodal content is deferred according to the attachment service contract rather than silently presented as if the child model could interpret it. This behavior moves the capability boundary into a testable runtime component instead of scattering provider-specific checks across the UI.

The child kernel uses the same core loop as the parent, but recursive sub_agent calls are disabled. The caller may select a read-oriented mode, full execution mode or an explicit allowed-tool set. If a tool requires approval, the child can enter a waiting state; an approval continuation appends the tool result to durable child history before resuming. Parent identifiers on child events allow the UI to display delegated work without conflating the child timeline with the parent turn.

This design has a modest interpretation. It is an integration mechanism that makes heterogeneous model use explicit and observable. It does not learn which model is optimal, estimate a universal capability score, or establish that delegation improves answer quality for arbitrary providers.

2.3. Durable state, control and recovery

LamTools creates checkpoints around model turns and represents checkpoint relationships as a graph. A restore can select a scope, such as the child conversation, without rewriting the parent conversation. A fork creates a new session while preserving a graph edge to the source checkpoint. The same state boundary supports cancellation persistence and approval continuation, so an interrupted or waiting child is not reduced to an in-memory callback.

Arrange adds scheduled and event-triggered jobs. The runtime records occurrences, handles pause and cancellation transitions, reclaims expired leases and checks stale active turns during startup recovery. These features matter because delegation in a desktop runtime is not only a one-shot function call: the operator must be able to stop, inspect, resume or branch a task. The tests validate these transitions using deterministic stores and clocks; they do not claim production-scale fault tolerance.

2.4. Version-bound evaluation overview

The evaluation separates deterministic runtime behavior from retrieval quality. The first gate asks whether the implementation preserves its stated state and permission transitions. The second is a small quality case study over a fixed corpus and golden questions. Figure 3 shows the recorded test counts and derived retrieval-score differences; it does not turn the two kinds of evidence into one aggregate score.

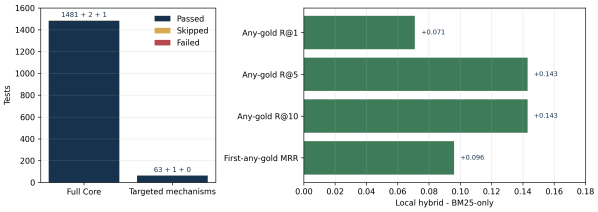

Figure 3. Evaluation overview. The left panel reports recorded test counts; the right panel reports the local-hybrid minus BM25-only change for stored retrieval metrics.

All values refer to LamTools v0.2.6, tag v0.2.6, commit dd54b7fdd57f4eb0926f1dd3a94fbf2c2bb0fd8a. The latest correctly configured full Core run collected 1,484 tests and recorded 1,481 passed, 2 skipped, one timing-sensitive failure and three warnings. The complete output is retained in docs/paper/evidence/core-pytest-v0.2.6-20260820.log. The failing assertion was the parallel-tool timing threshold; the same test passed in an isolated rerun retained at docs/paper/evidence/timing-sensitive-isolated-v0.2.6-20260821.log. The refreshed targeted mechanism run covered delegation, named sessions, model override, capability handling, checkpoint graph operations, rollback/fork and Arrange scheduling/recovery; it recorded 63 passed and 1 skipped test in docs/paper/evidence/targeted-mechanism-v0.2.6-20260821.log. The targeted run is a mechanism probe, not an independent replacement for the full suite.

Evidence stream	Unit of analysis	Recorded result	Interpretation
Full Core suite	Repository test cases	1,481 passed; 2 skipped; 1 timing-sensitive failure	Broad regression evidence, with one unresolved environment-sensitive test
Targeted runtime suite	Delegation, persistence and scheduling cases	63 passed; 1 skipped	Direct behavioral evidence for the paper's mechanism
Retrieval case study	14 questions over 8 documents, two retrieval modes	14 stored records per mode	Small, inspectable quality/latency comparison
Reproducibility target	Release and source state	v0.2.6 / dd54b7f...	Prevents mixing later code with reported values

2.5. Retrieval case study and trade-off

The repository contains an eight-document Chinese contract corpus and a 14-question golden set. The questions include exact, numeric, paraphrase and multi-document forms. The stored comparison is between BM25-only and local embedding plus hybrid retrieval. Local hybrid retrieval raises any-gold document Recall@1 by 0.071, Recall@5 and Recall@10 by 0.143, and first-any-gold MRR by 0.096. Warm query-time p95 latency rises by 4.92 ms, approximately 3.8 times the BM25-only value in the recorded environment.

Configuration	Any-gold Recall@1	Any-gold Recall@5	Any-gold Recall@10	First-any-gold MRR	Warm query p95
BM25-only	0.786	0.857	0.857	0.821	1.75 ms

Configuration	Any-gold Recall@1	Any-gold Recall@5	Any-gold Recall@10	First-any-gold MRR	Warm query p95
Local embedding + hybrid	0.857	1.000	1.000	0.917	6.67 ms
Difference	+0.071	+0.143	+0.143	+0.096	+4.92 ms

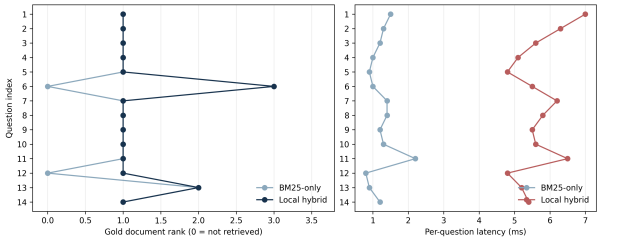

Figure 4. Per-question evidence from the stored retrieval reports. Rank zero means that the gold document was not retrieved; the latency panel preserves observed wall-clock values rather than a fitted distribution.

The per-question view makes the aggregate result less misleading. BM25-only misses the multi-document arbitration question and the paraphrase about deposit return, whereas local hybrid retrieval returns a gold document for both at a non-zero rank. The hybrid path also changes some top-document ordering and increases every recorded per-question latency. The case study supports a narrow engineering statement: the local vector leg improves coverage on this corpus at a measurable local latency cost. It does not support claims about legal-answer correctness, large-scale retrieval quality or provider-independent performance.

3. Discussion

The strongest technical signal is not that LamTools calls another model. It is that the call is made legible as a capability contract and a durable scope. The effective model, attachment capability, child-session identity, tool mode, approval state and parent projection can each be inspected, tested and recovered. This provides a runtime guard against a class of silent multimodal mismatches before they are misinterpreted as model failures; it is not a measured provider failure-rate reduction. The cost is additional state coordination and a larger failure surface than a single model call. A child session may wait for approval, require recovery or carry content that the selected model cannot consume; making those conditions explicit improves observability but does not remove them.

The retrieval case study illustrates the same trade-off at another layer. Adding a local vector leg improves first-hit coverage for this small corpus, but warm query-time p95 latency increases from 1.75 to 6.67 ms. The correct conclusion is not that hybrid retrieval is better in the abstract. It is that the runtime can expose a measurable quality/latency choice and preserve per-question evidence needed to revisit that choice.

For software-oriented agent systems, a credible paper should state which behavior is implemented, which behavior is planned, what baselines hold constant and where the evidence stops. LamTools is a compact case study of that discipline: its strongest result is an auditable connection between capability handling, durable control and measurable trade-offs, not evidence that a local runtime or delegation policy is universally superior.

4. Related work

General multi-agent frameworks such as AutoGen [1] and AgentScope [5] provide abstractions and platform support for building multi-agent applications. LamTools overlaps at the level of agent composition but makes a narrower runtime claim: its child is a durable tool invocation with a stable identity, bounded recursion and explicit model/capability handling. The paper does not compare task quality against these platforms or claim a new multi-agent protocol.

FrugalGPT studies cascades across language models for cost and quality [2], while RouteLLM learns routers from preference data [3]. These lines of work motivate heterogeneous model use, but LamTools does not train a router, learn a cascade policy or optimize a cost objective. Its evidence concerns the explicit execution path that allows an operator or configured call to select a child model and preserve the consequences in local runtime state.

MemGPT treats context management and interrupts as an operating-system-inspired memory problem [4]. LangGraph documents checkpoint savers, threads and durable graph state [8]. LamTools shares the engineering concern for resumable execution, but its evidence concerns its own SQLite checkpoint graph, approval continuation, child-session state and rollback/fork operations. MCP standardizes prompts, resources and tools for LLM applications [7]; LamTools integrates MCP within a broader local, approval-aware toolbox that also contains native and plugin-scoped tools. AgentDojo evaluates tool-using agents under prompt-injection attacks [6]; LamTools implements local approval and visibility boundaries, but does not provide a security proof or adversarial-robustness evaluation.

The local-first framing is grounded in the principles of local ownership, availability and user control described by Kleppmann et al. [9]. LamTools applies that framing to orchestration state in a desktop runtime; it does not claim that every configured provider call remains local.

Related line	What prior work provides	LamTools boundary
Multi-agent platforms	Agent and platform abstractions [1, 5]	Runtime-level child-session operation
Model routing	Cost/quality selection and learned routers [2, 3]	Explicit selection path; no learned optimizer
Stateful agents	Context management and interruption concepts [4]	SQLite runtime state and checkpoint graph
Durable graphs	Checkpoints, threads and savers [8]	Local approval, child scopes, rollback/fork
Tool security and interoperability	Tool protocols and adversarial evaluation [6, 7]	MCP plus native/plugin tools under local permissions; no security proof
Local-first software	Local ownership and user control principles [9]	Local orchestration state, not necessarily local providers

5. Methods

5.1. Audit and version control

The evidence boundary was established by reading the active core/ package, its tests, release workflow, configuration and documentation at v0.2.6. Archived products and roadmap-only features were excluded. Every paper-facing result is tied to tag v0.2.6 and commit dd54b7fdd57f4eb0926f1dd3a94fbf2c2bb0fd8a; the repository audit was performed on 20 August 2026 and the logged full rerun on 21 August 2026. The full Core run used the temporary LAMTOOLS_COMMAND_SHELL=pwsh selection because the default Git Bash environment could not resolve the Windows Python command; this is an environment-selection detail, not a missing Python installation.

5.2. Deterministic runtime evaluation

The runtime evaluation uses fake deterministic model clients and temporary SQLite workspaces. Tests assert state transitions and event projections rather than asking a provider to produce a subjective answer. The targeted set covers canonical model-ID resolution; named child-session reuse; model override; multimodal forwarding and text-model deferral; tool mode and allow-list behavior; recursive delegation blocking; approval continuation; cancellation persistence; parent/child event projection; checkpoint graph creation; scoped restore; rollback; fork; scheduled execution; pause/cancel transitions; lease reclaim; and startup recovery.

The full and targeted counts are reported as recorded test outcomes. Skips remain visible. No failed test was removed from the paper-facing evidence, and no provider-backed score was substituted for a deterministic behavioral assertion.

5.3. Retrieval protocol

The retrieval case study uses the repository's stored 14-question golden set, eight-document Chinese contract corpus and two report files: e2e/rag-eval/reports/retrieval-none-20260815-095234.json and e2e/rag-eval/reports/retrieval-local-20260815-101253.json. The same questions and gold-document labels are used in both modes. The reports retain per-question rank, top-document identifiers, vector-hit counts where applicable and wall-clock latency. The paper-facing derivation is stored in docs/paper/evidence/retrieval-derived-v0.2.6.json and is generated by docs/paper/derive_retrieval_metrics.py without rewriting the raw reports.

For a question set of size (N), any-gold document Recall@k is the fraction of questions for which any gold document appears in the first (k) deduplicated document results. Multi-document questions count as a hit when any gold document appears; complete-set recall is not measured. MRR is (N^{-1}sum_i 1/r_i), where (r_i) is the rank of the first any-gold document and a missing rank contributes zero. Warm query-time p95 is calculated by inclusive linear interpolation over the stored per-question timings. The reported differences are direct arithmetic differences between derived aggregate values; no confidence interval or significance test is claimed because the case study is small and the stored reports do not contain repeated independent runs.

5.4. Provider smoke-test protocol

The paper-facing claims remain deterministic, but a separate provider smoke test is prepared for the OpenCode Go endpoint [11]. The endpoint is OpenAI-compatible and uses /chat/completions for deepseek-v4-flash and mimo-v2.5. Each model receives the same fixed prompt with temperature zero and a small output cap. The smoke test records only model, HTTP status, response shape, latency and provider-reported usage; it intentionally does not retain generated text or credentials. These results may be reported as an implementation sanity check only, not as a benchmark or model-quality evaluation.

5.5. Reproducibility and release state

The reproducibility bundle includes manuscript sources, bibliography, figure generator, generated figures, evaluation harness, golden questions, raw JSON reports and methodology notes. The software target is the existing public GitHub Release v0.2.6; the historical tag is not amended. Provider credentials, private configuration and user data are excluded. The paper record should archive one combined Chinese-primary/English PDF, while the software record should archive the existing release separately.

6. Limitations and non-claims

The evaluation is small and mostly deterministic. No provider-backed model-quality comparison, external user study or large-scale benchmark was performed. The retrieval corpus has eight documents and 14 questions; the observed values should not be generalized to legal retrieval, other languages or larger corpora. The report files do not contain a complete machine-environment fingerprint, so the values are version-bound case-study evidence. Provider behavior, API latency, model versions, network conditions and local hardware can change the results.

Some RAG session-history paths are present but not treated as fully validated because an older integration check retains a stale expectation. Planned VLM format routing, complete table extraction and large-scale contract workflows are excluded. The system overlaps conceptually with prior agent runtimes, routing work, persistence systems and tool protocols; no priority or "first" claim is made. Finally, the paper has not undergone peer review and the two-author equal-contribution statement reflects author agreement, not a venue's editorial decision.

7. Data and code availability

LamTools is available at https://github.com/Lam-Arc/LamTools. The exact reproducibility target is the public v0.2.6 software release [10], commit dd54b7fdd57f4eb0926f1dd3a94fbf2c2bb0fd8a. The RAG harness, corpus, golden questions and raw reports are tracked under e2e/rag-eval/ in that release. The frozen software release is archived at Software DOI https://doi.org/10.5281/zenodo.22039646; this bilingual paper record is identified by Paper DOI https://doi.org/10.5281/zenodo.22040870.

8. AI assistance disclosure

Generative AI tools were used as auxiliary support during the preparation and review of this work. The research, technical decisions, interpretation, and final review were human-led.

9. Conclusion

LamTools is an implemented local-first agent runtime whose most defensible paper story is capability-aware child-agent delegation integrated with durable state, explicit tool boundaries and operator control. At the fixed v0.2.6 baseline, deterministic tests make delegation and recovery behavior inspectable, while the stored retrieval case study exposes a concrete quality/latency trade-off. The paper's value is therefore a reproducible system description with visible evidence boundaries, not an inflated claim of a universal routing method. A future release can expand provider-backed evaluation and external usage evidence; the present record remains valid when its exact version, limitations and non-peer-reviewed status stay visible.

References

1. Wu, Q., Bansal, G., Zhang, J., et al. (2023). AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. arXiv:2308.08155. https://doi.org/10.48550/arXiv.2308.08155

2. Chen, L., Zaharia, M., and Zou, J. (2023). FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance. arXiv:2305.05176. https://doi.org/10.48550/arXiv.2305.05176

3. Ong, I., Almahairi, A., Wu, V., et al. (2024). RouteLLM: Learning to Route LLMs with Preference Data. arXiv:2406.18665. https://doi.org/10.48550/arXiv.2406.18665

4. Packer, C., Wooders, S., Lin, K., et al. (2023). MemGPT: Towards LLMs as Operating Systems. arXiv:2310.08560. https://doi.org/10.48550/arXiv.2310.08560

5. Gao, D., Li, Z., Pan, X., et al. (2024). AgentScope: A Flexible yet Robust Multi-Agent Platform. arXiv:2402.14034. https://doi.org/10.48550/arXiv.2402.14034

6. Debenedetti, E., Zhang, J., Balunovic, M., et al. (2024). AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents. arXiv:2406.13352. https://doi.org/10.48550/arXiv.2406.13352

7. Model Context Protocol. (2025). Model Context Protocol Specification, protocol revision 2025-06-18. https://modelcontextprotocol.io/specification/2025-06-18/server/index

8. LangChain. (n.d.). LangGraph Persistence and Checkpointing. Official documentation. https://docs.langchain.com/oss/python/langgraph/persistence

9. Kleppmann, M., Wiggins, A., van Hardenberg, P., and McGranaghan, M. (2019). Local-first software: You own your data, in spite of the cloud. Proceedings of Onward! '19, 154-178. https://doi.org/10.1145/3359591.3359737

10. Zhang, Yulin, and Zhang, Yiming. (2026). LamTools (Version v0.2.6) [Computer software]. Zenodo. https://doi.org/10.5281/zenodo.22039646

11. OpenCode. (n.d.). Go documentation: endpoints, models and compatibility. Official documentation. https://opencode.ai/docs/go/